TRABALHO PRÁTICO • DESENVOLVIMENTO WEB / PWA

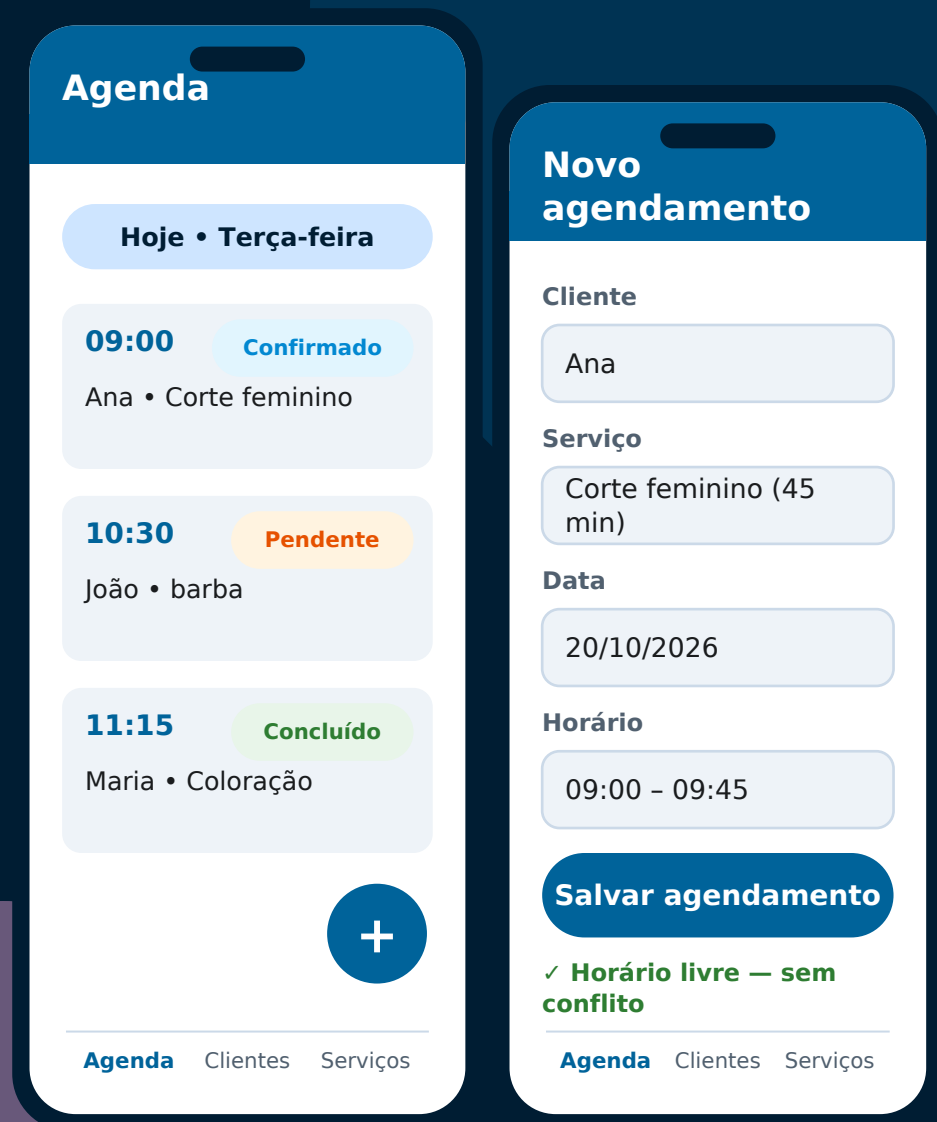
Agenda de Serviços

Agendamento inteligente para barbearias, salões, oficinas e prestadores de serviços

Grupo 2 — 4 integrantes

Integrantes: [Nome 1] • [Nome 2] • [Nome 3] • [Nome 4]
Turma/Disciplina: [preencher] • Professor(a): [preencher]

Apresentação • 15 minutos



O ponto de partida

O que foi pedido — e o que trazemos para apresentar

! A especificação do trabalho

• Desenvolver um aplicativo web em formato PWA, hospedado em nuvem
• Grupo 2: App Agenda de Serviços (barbearia/oficina) — cadastro de cliente, horários e status, com calendário
• Entregáveis: link público do app + link do repositório GitHub

✓ O que construímos

• Um app funcional de agenda de serviços, com o tema e as funções do Grupo 2
• Construído como Android nativo (Kotlin), não como página web
• Nesta apresentação, mostramos o que atende, onde diverge do PWA e como corrigir o rumo

Nosso compromisso nesta apresentação: demonstrar a solução, as decisões técnicas e o aprendizado — sem esconder as pendências.

Agenda manual gera retrabalho

Três dores de quem atende com hora marcada

1 Informação espalhada

Clientes, preços e horários ficam no caderno, na cabeça e nas conversas do WhatsApp.

Resultado: difícil consultar, buscar e manter o histórico.

2 Conflito de horário

Sem validar a duração de cada serviço, dois atendimentos acabam sobrepostos.

Resultado: espera, atraso em cadeia e cliente insatisfeito.

3 Esquecimento

Nem o profissional nem o cliente recebem um lembrete perto do atendimento.

Resultado: faltas (no-show) e horários vagos que poderiam render.

Público-alvo: barbearias, salões de beleza, oficinas e prestadores autônomos que precisam controlar atendimentos pelo celular — mesmo sem internet.

O que o app entregue faz

Funcionalidades ligadas diretamente às três dores

Clientes

Cadastro com telefone/WhatsApp, busca e edição.

Serviços

Preço em R$ e duração em minutos de cada atendimento.

Agendamentos

Cliente + serviço + data e horário, com observações.

Sem conflito de horário

Validação de sobreposição que bloqueia choque de agenda.

Visão da agenda

Lista por dia, busca em tempo real e filtros por status.

Lembretes locais

Notificação 15 minutos antes, mesmo com o app fechado.

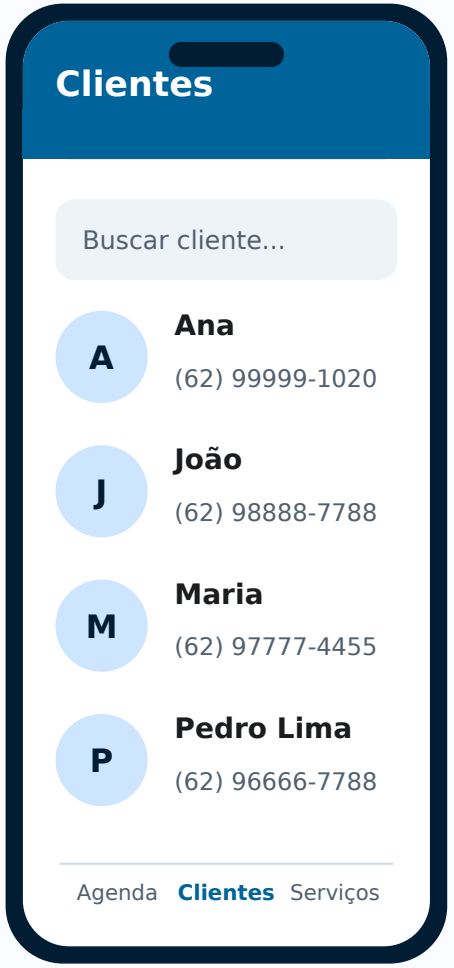
Ciclo de vida do atendimento: Pendente → Confirmado → Concluído ou Cancelado

Tudo salvo localmente no aparelho: o app funciona 100% offline (detalhes técnicos nos próximos slides).

Do cadastro ao lembrete

As telas principais e o fluxo que vamos mostrar ao vivo







Fluxo de uso

1. **Cadastrar cliente e serviço**
   Base reutilizável em todos os agendamentos.
2. **Escolher data e horário**
   O fim é calculado pela duração do serviço.
3. **Validar conflito**
   Sobreposição é bloqueada antes de salvar.
4. **Acompanhar a agenda**
   Buscar, filtrar por status e atualizar.
5. **Receber o lembrete**
   Notificação 15 min antes do atendimento.

A base técnica do app

Android nativo moderno: Kotlin + Jetpack Compose + Room

Interface

Kotlin 2.1.10
Jetpack Compose + Material Design 3
Navigation Compose (barra inferior)

Arquitetura

MVVM + Repository Pattern
Coroutines, Flow e StateFlow
4 ViewModels (1 por tela)

Dados offline

Room 2.8.5 sobre SQLite
KSP (geração de código do Room)
Consultas reativas com Flow

Recursos de sistema

AlarmManager + NotificationManager
BroadcastReceivers (alarme e reinício)
App Shortcuts no ícone do app

Por que essas escolhas?

- UI declarativa acelera a construção das telas
- Banco local = offline de verdade, sem servidor
- Estado reativo mantém agenda sempre atual
- Acesso direto a alarmes e notificações do sistema

Android nativo ≠ PWA web

Arquitetura MVVM e dados offline

Separação de responsabilidades do clique até o banco SQLite

Telas Compose
Agenda • Clientes
Serviços • Formulário

▶

ViewModels
Estado com StateFlow
Validações da tela

▶

Repositories
Regras de acesso aos dados

▶

DAOs Room
Consultas SQL
Cliente • Serviço • Agenda

▶

SQLite (Room)
Banco local no aparelho
100% offline

Modelo de dados (3 entidades)

Cliente — nome e telefone/WhatsApp
Serviço — nome, preço (R$) e duração (min)
Agendamento — cliente + serviço (chaves estrangeiras), início, fim, status e observações

Exclusão em cascata e índices mantêm o banco íntegro e rápido.

A regra que evita conflito (SQL no Room)

```
novoInício < fimExistente
AND novoFim > inícioExistente
```

Ignora agendamentos CANCELADOS e o próprio registro quando ele está sendo editado.

A evolução em 4 etapas

Cada etapa deixou o app utilizável antes de partir para a próxima

1

Interface e navegação

Design system em Jetpack Compose (Material 3) e navegação entre Agenda, Clientes e Serviços.

2

Clientes e serviços

Cadastros completos (CRUD) com persistência no Room Database.

3

Agendamentos

Calendário/data e hora, cálculo de fim pela duração e verificação de conflito.

4

Lembretes e acabamento

Notificações locais, atalhos do ícone do app e documentação (README).

Registro de transparência: no GitHub, essas etapas estão hoje consolidadas em um envio principal — melhorar a granularidade dos commits entrou para a nossa lista de correções.

Os 3 principais desafios

O que travou, como enfrentamos e o que ficou diagnosticado

1 Impedir sobreposição de horários

O problema

Serviços têm durações diferentes: comparar só o início deixa conflitos passarem.

Nossa resposta

Guardamos início e fim de cada atendimento e testamos a sobreposição de intervalos no SQL, ignorando cancelados.

2 Lembrar mesmo com o app fechado

O problema

Android restringe alarmes exatos (12+) e exige permissão de notificação (13+); reiniciar o celular apaga alarmes.

Nossa resposta

AlarmManager 15 min antes com fallback quando não há permissão de exato + BootReceiver para reagendar após reiniciar.

3 Fazer rodar fora da máquina de origem

O problema

Diagnóstico: local.properties apontando para C:\Users\CLIENTE\... foi versionado; .gitignore não era de Android; SDK 37/JDK 25 são exigências recentes.

Nossa resposta

Correção planejada: .gitignore de Android, remover arquivos locais/build do Git e testar a partir de um clone limpo (slide 12).

O que funcionou bem

Resultados que conseguimos Demonstrar

- ✓ Cadastros de clientes e serviços completos, salvos offline no aparelho
- ✓ Agenda reativa: salvou/alterou, a lista atualiza na hora (Flow/StateFlow)
- ✓ Conflito de horário bloqueado antes de salvar — sem dupla reserva
- ✓ Busca em tempo real e filtros por status do atendimento
- ✓ Lembrete automático 15 min antes, inclusive após reiniciar o celular
- ✓ Atalhos ao pressionar o ícone do app (App Shortcuts)

4 telas principais	**3** entidades no banco
4 status de atendimento	**15 min** antecedência do lembrete
API 24+ Android 7.0 ou superior	**100%** funciona offline

Leitura honesta: onde estamos frente ao PWA

Comparativo direto com a especificação da atividade

Requisito da atividade	Situação no projeto	Veredito
Tema do Grupo 2 (clientes, horários, status, agenda)	Funções implementadas no app	ATENDE
Código versionado no GitHub	Repositório publicado	ATENDE
manifest.json + Service Worker (offline web)	AndroidManifest.xml + banco Room (offline nativo)	NÃO ATENDE
Hospedagem (Vercel/Netlify/Pages) + link público	Sem deploy — roda via Android Studio/APK	NÃO ATENDE
LocalStorage/IndexedDB ou Firebase/Supabase	Room (SQLite) — equivalente funcional, outra tecnologia	NÃO ATENDE
Responsivo web + notas do Lighthouse	App Android, sem URL para auditar	NÃO ATENDE

Conclusão: a solução é funcional, mas equivalência nativa não gera os artefatos cobrados — manifest, Service Worker, URL pública e Lighthouse.

O que faríamos a seguir

De um app Android funcional a um PWA publicável e mensurável

A Corrigir o repositório atual

- .gitignore próprio de Android
- Retirar do Git: local.properties, build/, .gradle/ e .idea/
- Permissão de execução no gradlew e versões coerentes no README
- Commits pequenos por etapa e teste a partir de clone limpo

B Portar a solução para PWA

- Recriar as telas em web responsiva (mobile-first)
- IndexedDB com o mesmo modelo e a mesma regra de conflito
- manifest.json (ícones e atalhos) + Service Worker (cache offline e notificações)
- Deploy gratuito (GitHub Pages/Vercel) e metas do Lighthouse

1 Higiene do repo → **2 MVP web** → **3 Camada PWA** → **4 Deploy + Lighthouse**

O que levamos desta atividade

Aprendizados técnicos e de processo da equipe

1 Requisito antes da tecnologia

A especificação define o formato (PWA). Escolher a ferramenta antes de relê-la gerou o nosso maior desvio.

2 Offline é arquitetura

Persistência local, estado reativo e regras no banco foram decisões de projeto — não um detalhe no fim.

3 Versionamento também é engenharia

.gitignore correto, nada de caminho local no Git e commits pequenos valem tanto quanto código funcionando.

4 Testar como o avaliador testa

Clonar em máquina limpa e abrir pelo link público revelaria cedo os problemas de ambiente e de entrega.

Experiência da equipe: dividir o trabalho em frentes (interface, dados, agendamento e sistema/documentação) mostrou que integração e leitura crítica do requisito importam tanto quanto programar.

ENCERRAMENTO

Obrigado!

Perguntas?

Entregamos uma base funcional e offline de agenda de serviços.
O próximo passo é transformá-la, de fato, em um PWA publicável, instalável e mensurável.

Repositório no GitHub

github.com/LuizFelip908/Agenda-de-Servicos

✓ Código publicado

Link público: a portar

Demonstração: app Android (emulador/celular)

Grupo 2 — [Nome 1] • [Nome 2] • [Nome 3] • [Nome 4] | Agenda de Serviços — barbearia & oficina